PalmSource™ 99

Developing C/C++ Conduits for Windows®

Geoff Richmond
Sr. Development Support Engineer

3Com®
More connected.

Overview

PalmSource 99

- Synchronization Overview
- HotSync® Architecture
- Choosing the Right Development Tool
- Conduit Development
- Generic Conduit
- Installing Conduits
- Summary

Synchronization Overview

PalmSource 99

- Desktop Synchronization
  - Handheld and synchronization engine on local workstation
  - Data usually located on local workstation
- Network Synchronization
  - Handheld connected to any computer on network
  - Synchronization runs on handheld user's workstation
  - Data accessible from handheld user's workstation
- Server Synchronization
  - Handheld connected directly to network (or client)
  - Synchronization runs on servers
  - Data and synchronization engines distributed

Synchronization Dataflow







Mirror image	Ideal for applications that run on both platforms and allow modifications on both; makes data identical on both platforms. (address book and date book applications)	Your conduit must provide conflict resolution solutions when the same record has been modified on both platforms. This must occur without user intervention.
One directional Upload / Download	For applications that run on both platforms but only allow data modification on one; copies the data to the other platform.	Requires less time, and should be used when possible
Transaction based	For applications that need to perform additional processing on the desktop computer between record synchronizations. (e-mail, web, PQA, etc.)	Only use this when required, because it slows down the HotSync® process.
Back up	For applications that don't have PC components; simply copies the handheld data to the PC.	Can use the Backup conduit provided by Palm Computing.

HotSync® Architecture

PalmSource 99

HotSync Manager



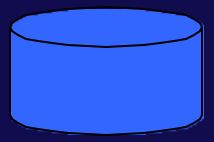
Conduit API
- OpenConduit()
- CfgConduit()

Application-Specific Conduit



Conduit Base Classes
- CSynchronizer
- CBaseMon

Data Source



- Local/networked data file
- Local/networked database (ODBC)
- Web-based data source
- E-mail or news server
- Dial-in connection

SyncManager API
- SyncOpenDb()
- SyncReadNextModified()
- SyncWriteRec()

Handheld



Contains applications/databases
- Unique creator id for each application
- Built-in HotSync device client
- HotSync process initiated by the device

Conduit Development Environments

- CDK 3.0 for Windows®
  Palm Computing®
- CDK 2.2 for Macintosh®
  Palm Computing
- CDKJ for Java®
  Palm Computing
- Satellite Forms™
  Puma Technology®
- Scout Architect
  Riverbed Technologies®



http://www.palm.com/devzone/conduits.html





Features of Palm OS® Conduit Development Kits

PalmSource 99

- Palm Conduit Wizard
  - Project Starter Code Generation
- SyncManager API
  - Database/record access
- Conduit Base Classes
  - Core synchronization logic
- Conduit Sample Projects
  - Source code for Palm standard applications

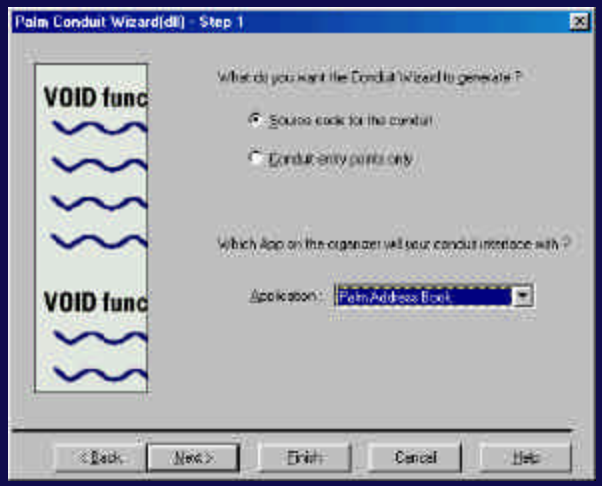
Features of Palm OS® Conduit Development Kits

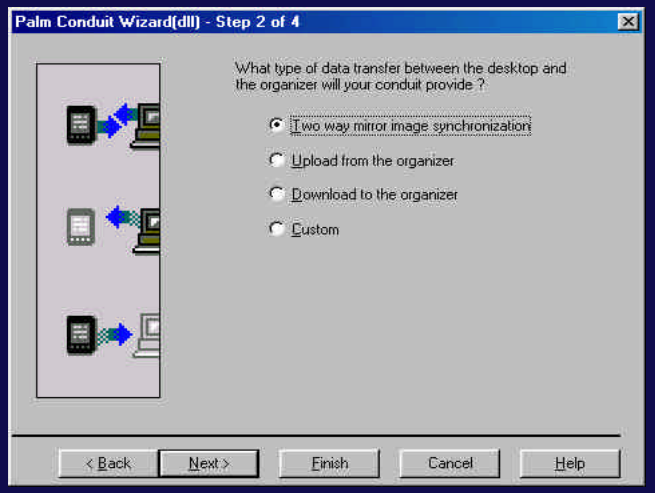
PalmSource 99

- Install Aide API
  - Permits easy installation/management of Palm Computing Platform applications.
- Conduit Manager API
  - Permits easy installation/management of conduits.
- User Manager API
  - Permits easy management of Palm users.
- HotSync® API (HSAPI)
  - Programmatic control of HotSync technology process

Palm OS® Conduit Wizard

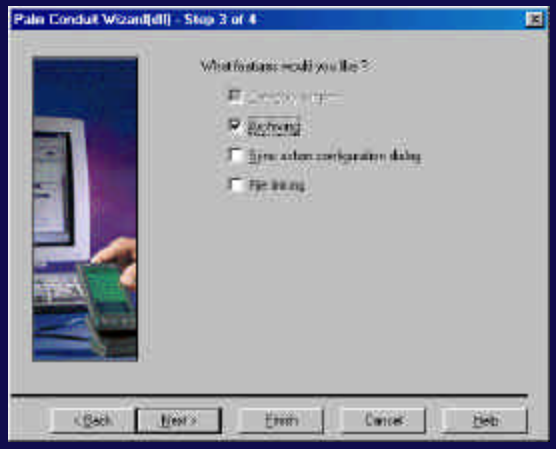
PalmSource 99





Easy configuration of conduit starter code

- Microsoft® Visual C++ 5.0
- Generates conduit base classes
- Generic Conduit supported in CDK 4.0



Conduit Entry Points

PalmSource 99

Required

- OpenConduit()
  - Synchronization entry point
- GetConduitName()
  - Returns name of conduit for display purposes
- GetConduitVersion()
  - Returns conduit version
- GetConduitInfo()
  - Returns conduit name, MFC version, or default sync action

Optional

- CfgConduit()
  - Display any UI necessary to configure conduit
  - ConfigureConduit()
    - (HotSync 2.0 or earlier)
- ConfigureSubscription()
  - Display any UI necessary to configure file linking
- SubscriptionSupported()
  - Is file linking supported by this conduit
- ImportData()
  - Configure field mapping and import file link data

SyncManager API

Database/Record Structure

Database Name
Database Creator
Database Type
Number of Records
Last Sync Date
AppInfo Block Pointer
Sort Info Pointer
Data Record 1
Data Record 2
Data Record n

Use SyncManager API to access remote databases and records.

SyncReadNextModifiedRec()

CRawRecordInfo Class

m_RecId
m_RecIndex
m_attribs
m_CatId
m_pBytes



Unique Id	Category	Attribs	Raw Record Data

Using “Palm™ Desktop Conduit” Base Classes

PalmSource 99

Creating a Conduit from CBaseMon/CBaseTable

1. Derive CMyRecordSchema from CBaseSchema

The schema tells the conduit base classes about your record format. Describe your record in DiscoverSchema()

```
long CMemoSchema::DiscoverSchema(void)
{
        m_FieldsPerRow = 6;
        m_FieldTypes.SetSize(m_FieldsPerRow);
        m_FieldTypes.SetAt(mpFLDRecordID,   (WORD)eInteger);
        m_FieldTypes.SetAt(mpFLDStatus,     (WORD)eInteger);
        m_FieldTypes.SetAt(mpFLDPosition,   (WORD)eInteger);
        m_FieldTypes.SetAt(mpFLDPrivate,    (WORD)eBool);
        m_FieldTypes.SetAt(mpFLDCategoryID, (WORD)eInteger);
        m_FieldTypes.SetAt(mpFLDMemo,       (WORD)eString);

        // Be sure to set the 3 common fields' position
        m_RecordIdPos     = mpFLDRecordID;
        m_RecordStatusPos = mpFLDStatus;
        m_CategoryIdPos   = mpFLDCategoryID;
        m_PlacementPos    = mpFLDPosition;

        return(0);
}
```

Using "Palm™ Desktop Conduit" Base Classes

PalmSource 99

Creating a Conduit from CBaseMon/CBaseTable (Cont.)

2. Derive CMyRecord from CBaseRecord

   Performs all record manipulation. Over-ride all functions for record creation, comparison, and setting and getting individual record fields.

3. Derive CMyTable from CBaseTable

   Main table of records from a database. Over-ride AppendDuplicateRecord() to facilitate adding your record to the table of records.

4. Derive CMyDTLinkConverter from CBaseDTLinkConverter

   Performs all data conversion to/from the handheld data format. Over-ride ConvertToRemote() and ConvertFromRemote() functions

5. Derive CMyMon from CBaseMon

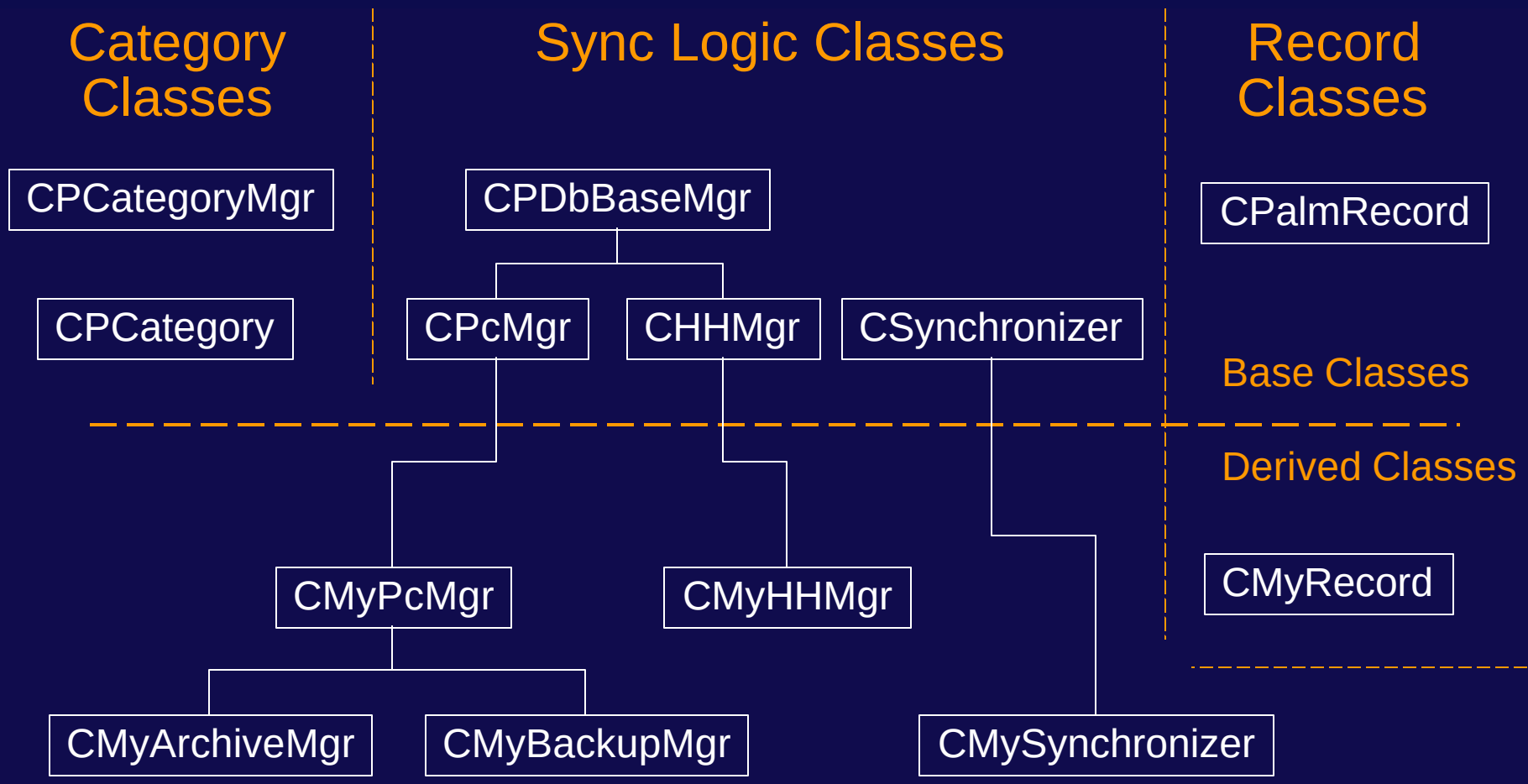
   Performs all record synchronization. Over-ride some functions that need to refer to CMyRecord.

Introduction to “Generic Conduit”

- No MFC classes used
  - Ideal for cross-platform conduit development
  - Debug conduits with release version of HotSync.exe
- Full source code provided
  - Permits easy configuration management
- Well-designed base classes
  - Permits maximum flexibility and code leverage
  - Easily modified to support CSV files, SQL, ODBC data sources, and other data formats

“Generic Conduit” Class Hierarchy

PalmSource 99



Conduit Control Flow

PalmSource 99

- HotSync® Manager calls your OpenConduit() function
  - Examine SyncProperties structure for sync information
- Your OpenConduit() function instantiates a CSynchronizer object and performs the following tasks:
  - Register the conduit
  - Create the Manager objects (handheld, desktop, archive)
  - Synchronize categories information using Native Sync logic
  - Synchronize application information (AppInfo block)
  - Synchronize records using Fast Sync, Slow Sync, HH overwrites desktop, Desktop overwrites HH, or No Action
  - Delete the Manager objects
  - Log results to HotSync log

Using "Generic Conduit" Base Classes

PalmSource 99

Creating a Conduit from Generic Conduit

1. Create your record class (CMyRecord) and a C-style record structure.

The generic conduit supports records as C++ classes or structures. All data elements should be declared public for easy access. Implement the Reset() method.

```
class CMemoRecord
{
public:
    CMemoRecord(void);
    ~CMemoRecord();

public:
    DWORD m_dwRecordID;
    DWORD m_dwStatus;
    DWORD m_dwPosition;
    DWORD m_dwPrivate;
    DWORD m_dwCategoryID;
    CPString m_csMemo;

public:
    void Reset(void);
};
```

```
typedef struct MemoRecord
{
    DWORD dwRecordID;
    DWORD dwStatus;
    DWORD dwPosition;
    DWORD dwPrivate;
    DWORD dwCategoryID;
    char  szMemo[MAX_MEMO_SIZE];
} MEMORECORD;
```

Schema (optional)

```
// Required for MFC Serialize data sources

typedef struct MemoSchema {
    DWORD lResourceID;
    DWORD lFieldsPerRow;
    DWORD lRecordIdPos;
    DWORD lRecordStatusPos;
    DWORD lPlacementPos;
    WORD wFieldCounts;
    WORD wFieldArray[mpFLDCount];
} MEMOSCHEMA;
```

Using “Generic Conduit” Base Classes

PalmSource 99

Creating a Conduit from Generic Conduit

2. Create CMyPcMgr as a sub-class of CPcMgr and implement the StoreDB() and RetrieveDB() methods.

The generic conduit permits easy customization of the data access routines. The sample conduits provide routines to read/write data in MFC serialize format.

The CPcMgr base class provides all record access routines. You sub-class it to read / write your record format.

```
long CMemoPcMgr::RetrieveDB(void)
{
   m_bNeedToSave = FALSE;
   if (!_tcslen(m_szDataFile))
      return GEN_ERR_INVALID_DB_NAME;

   if (m_hFile == INVALID_HANDLE_VALUE)
      return GEN_ERR_INVALID_DB;

   BOOL bDone = FALSE;
   long retval = 0;

   retval = CreateDBInfo();

   if (retval){
      CloseDB();
      return retval;
   }

   // Insert code to read records

   CloseDB();
   m_bNeedToSave = FALSE;
   return retval;
}
```

Using “Generic Conduit” Base Classes

PalmSource 99

Creating a Conduit from Generic Conduit

3. Create CMyBackupMgr and CMyArchiveMgr(s) as sub-classes of CMyPcMgr.

   Performs the Storing and Retrieving of backup and archive files.

4. Create CMySynchronizer as a sub-class of CSynchronizer and implement the Create...() methods and the AppInfo synchronization methods.

   Performs all Palm record synchronization. Over-ride all functions for creating the Manager Classes (CMyPcMgr, CMyHHMgr, etc.). Over-ride all functions for synchronizing ‘extra’ AppInfo data. (ie - any AppInfo data beyond category data.

Conduit Debugging Hints

PalmSource 99

- The source code for all conduit base classes is provided in C:\Cdk\Src. Use it to assist in debugging by compiling into your project. Remove it for final release
- Visit the Knowledge Base for debugging procedures. (http://www.palm.com/devzone)
- Do not mix release and debug dll's when debugging
- Use your derived record class, schema class, and table class in your MFC application
- See Conduit Guide for more tips

Choosing the Right Starting Point

PalmSource 99

	SyncManager API	Sample Code for Built-in Palm Applications	Generic Conduit	Form-Based Rapid Development Tool
Fast Prototype Application Development	Good	Good (Use Conduit Wizard)	Good (Use Conduit Wizard)	**Best**
Upload/Download Data Transfer	**Best**	Good	Good	**Best**
Interfacing to Standard Data Sources (Access, ODBC, CSV, etc.)			**Best**	**Best**
MFC-based Application		**Best**	Good	
Cross-Platform Source Code			**Best**	
Online/Offline Transaction Processing	**Best**			

Installing Conduits

PalmSource 99

- Use Palm-supplied API's for conduit and application installation
  - Install Aide API (PltInstallFile())
  - Conduit Manager API
  - User Manager API (UserData.dll)
- See sample install code on Knowledge Base

Summary

- The Palm Computing® Platform is the leader in handheld computing and information management
- Synchronization (Data Exchange) plays an important role in handheld computing; (personal and enterprise information management)
- Successful implementations depend on choosing the right synchronization strategy
- The HotSync® technology is adaptable to many synchronization scenarios
- The Generic Conduit provides a flexible framework for development of C++ Conduits for Windows
- The Palm Computing CDK provides a complete set of tools for conduit development for Windows


PalmSource™ 99
3Com
More connected.